

Plan

FinSight India — 6-Week Agentic AI + RAG Plan (Cash Equity + F&O)

An agentic research assistant over NSE/BSE filings, earnings concalls, derivatives data, and SEBI regulations — covering **both cash equities and F&O from week 1**, restructured around the order from the post you shared: **tool calling → agent loop → RAG → memory → evals → guardrails + deploy**.

This order is right for your background. Building the agent loop *before* RAG forces you to understand orchestration on its own terms, instead of RAG and agents getting tangled together from day one. By week 3, retrieval slots in as just another tool the agent already knows how to call.

Stack (all free): LLM: Google Gemini API (primary) + Groq (fast/backup) · Orchestration: LangGraph · Embeddings: sentence-transformers (local) · Vector DB: Chroma (local) · Reranker: local cross-encoder · Web search: Tavily free tier · Cash equity data: yfinance with [.NS/.BO](#) tickers · F&O + macro data: [jugaad-data](#) or [nsepython](#) (option chains, OI, bhavcopy, RBI repo rate) · Filings/concalls: NSE/BSE corporate announcements pages + company IR pages · Regulations: SEBI circulars (sebi.gov.in, free/public) · Eval: RAGAS · Deploy: Streamlit Community Cloud / HuggingFace Spaces

Two things that touch every week: report figures in ₹ crore/lakh, not billions, since that's what you'll be feeding the model and what it should echo back. And NSE's site has anti-scraping protection ([jugaad-data/nsepython](#) build in caching/throttling to handle this) — the retry/backoff habit matters for market-data tools, not just LLM rate limits.

Rhythm for 15+ hrs/week: roughly 2.5–3 hrs/day, 6 days/week. Treat one day as a "break it and fix it" day — debugging is most of the actual learning here.

Week 1 — Call an LLM API + Tool Calling

Goal: Get comfortable with the raw mechanics of function/tool calling, across both cash equity and F&O data sources, before any agent abstraction touches it.

Concepts required to know: message format and roles; tool/function schemas (JSON schema for parameters); how the model "decides" to call a tool vs. respond directly; parsing tool-call responses and feeding results back; basic F&O vocabulary you'll need for the tools below (option chain, open interest, strike, expiry, lot size).

Practical tasks / milestones: A single Python script, no framework. Build 5 tools:

- `get_stock_price(ticker)` — yfinance, `.NS` tickers (e.g. `TCS.NS`)
- `get_company_overview(ticker)` — fundamentals via yfinance
- `get_option_chain(symbol)` — via `jugaad-data/nsepython`, for index or stock F&O (e.g. NIFTY weekly chain)
- `get_repo_rate()` — current RBI repo rate via `jugaad-data`'s RBI module
- `calculate(expression)` — simple safe calculator

Manually wire the round trip for each: model proposes a call → you execute the function → result goes back to the model → model answers. Do this with **both** Gemini and Groq so you see tool-calling is provider-agnostic, just with slightly different JSON shapes.

Day to day:

- Day 1–2: Get both API keys, make a first plain-text call to each, understand request/response shape.
- Day 3–4: Read each provider's function-calling docs, define schemas for the 3 cash-equity/utility tools, get the model to *propose* a call (don't execute yet).
- Day 5: Wire up execution + round-trip for those 3 tools.
- Day 6: Add `get_option_chain` and `get_repo_rate` the same way — same pattern, new data source.
- Day 7: Full end-to-end test of all 5 tools with both Gemini and Groq.

References: Gemini/Groq function-calling docs, `jugaad-data/nsepython` docs for the option-chain and RBI endpoints, Anthropic's "Building Effective Agents" post (provider-agnostic framing, worth reading now).

Main output of the week: A working script with 5 tools spanning cash equities, F&O, and macro context, each independently tested end-to-end with two different LLM providers.

Week 2 — Add a Loop = Your First Agent

Goal: Turn the one-shot tool calls into a loop that chains multiple tools across *both* domains and reasons between them, without you scripting the order.

Concepts required to know: the ReAct pattern (Reason → Act → Observe, repeat); stopping conditions (max iterations, "final answer" detection); handling a tool that fails or returns nothing; state management across loop iterations.

Practical tasks / milestones: Introduce LangGraph — explicit state graphs save real pain once memory, RAG, and guardrails are all touching the same loop later. Port week 1's 5 tools in as LangGraph tool nodes. Target query: *"Is TCS's P/E higher than Infosys's, and are there unusual options activity building up in NIFTY this week?"* — this needs 4+ tool calls across cash and F&O data, with the model deciding the order itself.

Day to day:

- Day 1–2: LangGraph quickstart — nodes, edges, state.
- Day 3–4: Port all 5 tools in, build the loop with a max-iteration safety cap.
- Day 5: Deliberately break it — ask something needing a tool you haven't built, confirm it fails gracefully instead of hallucinating.
- Day 6–7: Add step-by-step logging (tool, args, result) — you'll reuse this in week 5 for evals.

References: LangGraph docs/tutorials, the ReAct paper (skim — the core idea is short).

Main output of the week: A LangGraph agent that autonomously chains tool calls across cash equity and F&O data sources to answer a multi-part question, with logging and a safety cap on loop length.

Week 3 — Add RAG

Goal: Ground answers in real documents — company filings/concalls *and* SEBI regulatory text — instead of only live numbers.

Concepts required to know: chunking strategy for messy financial documents (tables, scanned PDFs, footnotes); embeddings; vector similarity search; hybrid retrieval (keyword + embedding); reranking; basic metadata filtering (by doc type, company, date).

Practical tasks / milestones:

1. Pull quarterly results PDFs and concall transcripts for 5–10 NSE-listed companies (e.g. TCS, Infosys, Reliance, HDFC Bank, ICICI Bank, Tata Motors) from NSE/BSE corporate announcements or company IR pages.
2. Pull SEBI circulars relevant to F&O regulation (margin rules, lot-size changes, expiry-day rules) from sebi.gov.in.
3. Chunk and embed both sets locally (sentence-transformers), store in Chroma with a `doc_type` metadata field (`filing` vs `regulation`).
4. Add hybrid retrieval + a local cross-encoder reranker.
5. Wrap as two tools: `search_filings(query, ticker, fiscal_period)` and `search_regulations(query)` — give both to the week 2 agent.

Target query: *"What did Reliance's Q3 concall say about Jio's subscriber growth, and has SEBI changed F&O lot-size rules recently?"* — needs both tools, and the agent should pick correctly on its own.

Day to day:

- Day 1–2: Download and chunk company filings/concalls (scanned PDFs and inconsistent layouts will eat time here — budget for it).
- Day 3: Download and chunk SEBI circulars (cleaner text, faster to process).
- Day 4: Embed + load both sets into Chroma with metadata tagging.
- Day 5: Basic retrieval, sanity-check results manually across both doc types.
- Day 6: Add hybrid search + reranking.
- Day 7: Wire both search tools into the week 2 agent, test multi-hop queries spanning filings, regulations, and live data.

References: A RAG-from-scratch tutorial (LangChain or LlamaIndex), NSE/BSE corporate filings pages, sebi.gov.in circulars.

Main output of the week: An agent that retrieves and cites from two distinct grounded sources — company filings/concalls and SEBI regulatory text — choosing the right one per query, on top of the existing cash/F&O tool layer.

Week 4 — Add Memory

Goal: Stop the agent from forgetting what it's tracking — across both equity positions and F&O instruments — between turns.

Concepts required to know: short-term memory (conversation buffer within a session); longer-term/session state (a persisted watchlist); context window management (summarizing old turns instead of keeping everything verbatim).

Practical tasks / milestones: Multi-turn handling where the watchlist can mix instrument types:

- User: *"Track HDFC Bank, ICICI Bank, and NIFTY weekly options."*
- User (later): *"How are their margins and OI trends looking?"* → agent resolves "their" from memory and correctly applies margin to the banks and OI to the NIFTY options, not the other way around.

Persist the watchlist to SQLite or JSON so it survives between sessions. Add summarization once conversations get long enough to risk context limits — a good point to also stress-test your retry/backoff logic under realistic multi-turn load.

Day to day:

- Day 1–2: Add conversation buffer memory in LangGraph.
- Day 3–4: Add persisted, mixed-instrument-type watchlist state with file storage.
- Day 5: Add summarization for long conversations.
- Day 6–7: Stress-test a 15+ turn conversation mixing equity and F&O references, confirm correct resolution and that rate-limit handling holds.

References: LangGraph's memory/persistence docs.

Main output of the week: A multi-turn session that maintains a persisted, mixed cash-equity-and-F&O watchlist with correct pronoun/reference resolution across turns.

Week 5 — Add Evals

Goal: Prove the agent works across all its domains — cash, F&O, filings, and regulations — instead of relying on "it felt right when I tested it."

Concepts required to know: building a held-out eval set; RAG-specific metrics (faithfulness, context precision/recall) via RAGAS; agent-level metrics (task success rate, tool-call correctness, did it pick the right tool/source); latency and cost tracking.

Practical tasks / milestones: Write 25–35 questions spanning all five tool types (live price, fundamentals, filings/concall search, option chain/OI, SEBI regulation search) plus multi-hop combinations, with known correct answers/expected sources. Run RAGAS separately on the filings corpus and the SEBI corpus. Score tool-call correctness from your week 2 logs. Find and fix your worst 3 failure modes — pay particular attention to whether the agent ever confuses a regulation answer with a filing answer, since that's the most India-specific failure mode this build can have.

Day to day:

- Day 1–2: Write the eval set, covering all five tool types and a few deliberately ambiguous multi-hop questions.
- Day 3–4: Run RAGAS on both retrieval corpora, interpret scores.
- Day 5: Score tool-call correctness from logs.
- Day 6–7: Fix the worst failure modes, re-run to confirm improvement.

References: RAGAS documentation.

Main output of the week: A scored eval report covering cash, F&O, filings, and regulatory query types, with the top failure modes identified and at least partially fixed.

Week 6 — Add Guardrails + Deploy

Goal: Make it trustworthy enough to show someone, and put it behind a public URL.

Concepts required to know: citation enforcement (no claim without a source); refusing gracefully when retrieval comes back empty; input validation; cost/rate-limit ceilings so a runaway loop can't drain your free-tier quota; why F&O outputs specifically need a clear "informational, not trading advice" framing given how much retail F&O activity SEBI itself has flagged concerns about.

Practical tasks / milestones:

1. Enforce citations on every filings- and regulation-based answer; if nothing relevant is retrieved, the agent says so instead of guessing.
2. Add an explicit disclaimer whenever the agent surfaces option-chain/OI data, framing it as research output, not a trade recommendation.
3. Add a hard cap on loop iterations and tool calls per query.
4. Build a Streamlit UI with two views: a cash-equity research view and an F&O/OI view.
5. Deploy on Streamlit Community Cloud or HuggingFace Spaces.
6. Write a README with an architecture diagram (tools, agent loop, dual RAG sources, memory layer) — this is what a recruiter or interviewer actually looks at.

Day to day:

- Day 1–2: Citation enforcement + graceful "no answer" handling.
- Day 3: F&O disclaimer logic + iteration/cost caps.
- Day 4–5: Streamlit UI (both views).
- Day 6: Deploy.
- Day 7: README + architecture diagram + final end-to-end test pass.

References: Streamlit deployment docs, HuggingFace Spaces docs.

Main output of the week: A deployed, publicly accessible app handling cash equity and F&O research with enforced citations, a trading-advice disclaimer where relevant, and documentation suitable for a portfolio.

Optional Week 7 — Multi-Agent Stretch

Goal: Add a verification layer without starting a new project.

Concepts required to know: multi-agent coordination patterns, critic/verifier agent design, claim-checking against retrieved sources.

Practical tasks / milestones: Split the single agent into a "research agent" that drafts the answer and a "critic agent" that checks every claim against retrieved filings/regulations/data before the answer ships. Build with LangGraph subgraphs or CrewAI.

Day to day:

- Day 1–3: Design the research/critic split and the handoff contract between them.
- Day 4–5: Implement the critic agent's claim-checking logic.
- Day 6–7: Test on the week 5 eval set, compare accuracy with and without the critic layer.

References: LangGraph multi-agent/subgraph docs, CrewAI docs.

Main output of the week: A two-agent pipeline where a critic verifiably catches at least some unsupported claims the single-agent version missed.

Week 1 Notes

Function Calling with Gemini

[LINK](#)

Overview

Function calling helps connect external systems with other applications and APIs, making language models significantly more actionable. It enables models to interact with the outside world by generating structured data that represents a function call. Gemini provides instructions (JSON) to run specific functions with specific inputs rather than just generating text. To suggest a few examples, database search and retrieval, controlling smart home devices and booking flights through natural language.

How it Works

To implement function calling, developers define function declarations which act as blueprints for Gemini.

1. **Define:** Provide declarations including names, descriptions, and parameters.
2. **Analyze:** Gemini determines if a request requires a function call.
3. **Generate:** Gemini outputs the necessary JSON for the call.
4. **Execute:** The developer's code executes the function using the provided inputs.

Key elements of a function declaration

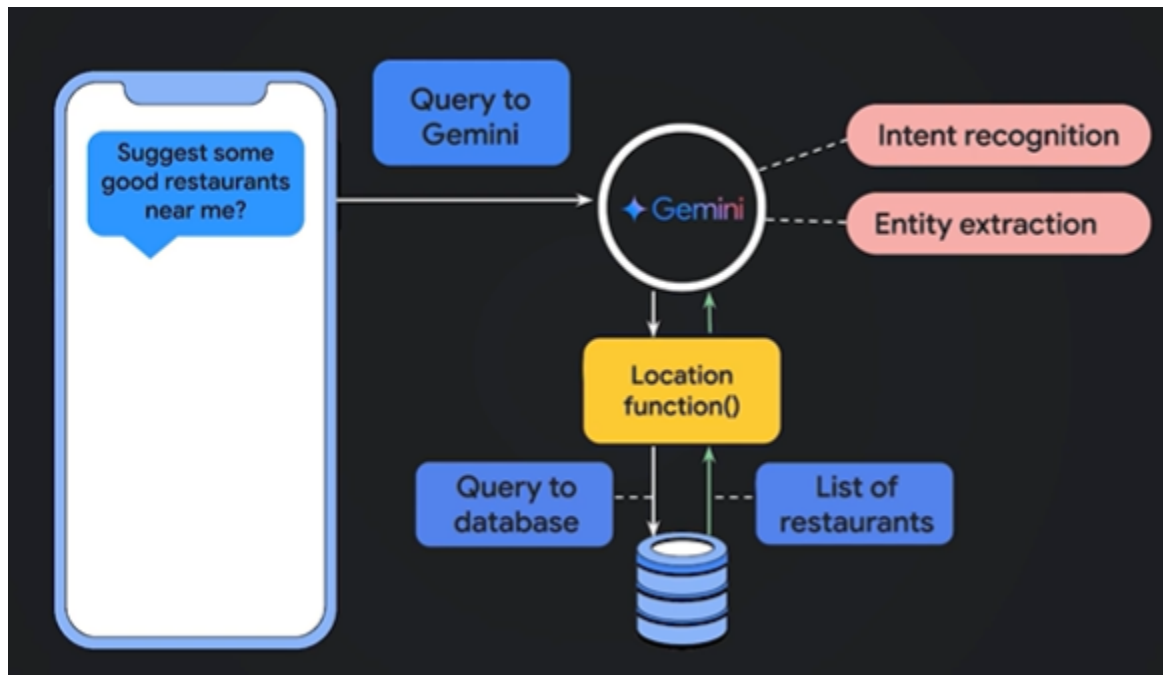
```
add_to_database = genai.protos.FunctionDeclaration(  
    name="add_to_database",  
    description=textwrap.dedent("""  
        Adds entities to the database.  
        """),  
    parameters=genai.protos.Schema(  
        type=genai.protos.Type.OBJECT,  
        properties = {  
            'people': people,  
            'places': places,  
            'things': things,  
            'relationships': relationships  
        }  
    )  
)
```

An important note, Gemini doesn't actually execute the functions. You need to write code on your end to do that. Gemini's job is to figure out which function to call and what inputs to provide.

Advanced Interaction:

Function calling is highly effective in multi-turn conversations, allowing Gemini to use the result of one function call as context for subsequent requests, creating a dynamic and interactive experience. Gemini can make a function call, get the result, and then use that result in the next turn of the conversation. This allows for very dynamic and interactive experiences.

Eg: Asking Gemini to find restaurants near you. It could call a function to get your location, then use that location to search a restaurant database, and then finally present you with a list of options.



Example: Get Weather

```
from google import genai

weather_function = {
    "type": "function",
    "name": "get_current_temperature",
    "description": "Gets the current temperature for a given location.",
    "parameters": {
        "type": "object",
        "properties": {
            "location": {
                "type": "string",
                "description": "The city name, e.g. San Francisco",
            },
        },
        "required": ["location"],
    },
}

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.5-flash",
    input="What's the temperature in London?",
    tools=[weather_function],
)

for step in interaction.steps:
    if step.type == "function_call":
        print(f"Function to call: {step.name}")
        print(f"Arguments: {step.arguments}")
```

1. Define the function → You provide Gemini with a description of the available function. This includes: function name, purpose (description), input parameters and their types. This is only metadata - not an actual Python function.
2. Send the User Prompt and Available Tools → The user's query and the list of available tools are sent to Gemini.
3. Gemini Decides Whether a Tool is Needed → If the question requires external information, Gemini does not answer directly. Instead, it returns a function call such as:

```
json
{
  "name": "get_current_temperature",
  "arguments": {
    "location": "London"
  }
}
```

```
}
```

The model is only requesting the function call—it does not execute it.

4. Your Application Executes the Function → Your code receives the function call and invokes the corresponding Python function or API.

```
def get_current_temperature(location):  
    # Call a weather API  
    return {  
        "temperature": 22,  
        "unit": "C" }  

```

5. Return the Result to Gemini → Your application sends the function's output back to the model.

```
json  
{  
    "temperature": 22,  
    "unit": "C"  
}
```

6. Gemini Generates the Final Response → Using the returned data, Gemini produces a natural-language answer. Example: "The current temperature in London is 22°C."

Overall Flow

User Query



Gemini receives prompt + available tools



Gemini determines if a tool is required



Returns a function call with arguments



Your application executes the function/API



Returns the result to Gemini



Gemini generates the final user-facing response

How a model decides to call a tool vs. respond directly

When you send a request to the model, you also send a list of tools it's "allowed" to use — each with a name, a description, and what inputs it needs. The model reads your question, reads those tool descriptions, and essentially asks itself: "Do I already know this, or do I need fresh/real data to answer correctly?"

- "What is TCS's current stock price?" → the model knows it doesn't have live prices baked into its training, sees `get_stock_price` is available and matches, and requests that tool instead of guessing a number.
- "What is a P/E ratio?" → this is a plain concept question, no live data needed, so it just answers directly in text.

The model(LLM those API is being used) has been trained on tons of examples of question → matching tools, so it pattern-matches your question against the tool descriptions you gave it. This is why your tool docstrings matter so much - a vague description makes the model's decision worse, a precise one makes it reliable. This is also exactly why you sometimes reach for the calculation on $2+2$ – the tool existed and loosely matched, so it used it, even though it didn't need to.

F&O Vocabulary

1. Option chain — a table listing every available option (Call and Put) for a stock/index at different price levels, for a given expiry date. It's literally a "chain" of strike prices with their prices, volume, etc.
 2. Open interest (OI) — the total number of option contracts currently active (not yet closed out or expired) at a given strike. High OI = lots of traders have positions there, often signaling a price level the market is watching closely.
 3. Strike — the fixed price at which an option lets you buy (Call) or sell (Put) the underlying stock/index. E.g. a "24300 Call" gives the right to buy at ₹24,300.
 4. Expiry — the date the option contract stops existing. After this date it's either exercised or worthless.
 5. Lot size — options aren't traded 1 unit at a time; they're traded in fixed bundles (e.g. NIFTY might trade in lots of 25). You can't buy 1 option — you buy 1 lot, which represents a fixed quantity.
-

Provider-specific tool-calling shapes (Gemini vs. Groq)

Both do the same three-step dance — model requests a tool, you run it, you send the result back, but the actual data shape differs in three specific spots:

1. How you describe the tools to the model (the schema): Gemini wants a flat list of tool definitions. Groq (OpenAI-style) wants each one wrapped in an extra envelope: ``{"type": "function", "function": {...}}``. Same information, just nested one level deeper for Groq.
2. How you read what the model wants to call: Gemini hands you a ready-to-use dictionary of arguments. Groq hands you the arguments as a plain text string that looks like JSON, you have to manually convert it into a dictionary yourself before you can use it (``json.loads(...)``).
3. How you send the tool's result back to the model: Gemini expects it tucked inside a user message. Groq expects a separate message with ``role: "tool"``, and it must be tagged with an ID matching exactly which tool call it's answering (since a model can request multiple tools in one breath, the ID says "this result answers request #2 specifically").

None of this changes the concept — it's the same round trip either way. Both "deliver a package" but use different forms.

Retry/backoff as a concept

APIs (Gemini, Groq, NSE, anything) put a ceiling on how many requests you can make in a given time window — your rate limit. When you go over it, instead of your data, you get an error (Gemini's was ``429 RESOURCE_EXHAUSTED``).

"Retry with backoff" means: instead of crashing the moment that happens, your code catches the error, waits a bit, then tries again — and if it fails again, it waits **longer** before the next attempt (that increasing wait time is the "backoff" part — you back off more each time instead of hammering the API repeatedly, which would just make things worse).

Concretely in ``agent.py``: when a 429 comes back, the code checks if the API told you exactly how long to wait (``retryDelay``), waits that long (or a default ~15s if not told), then retries — up to 4 attempts before finally giving up and showing you a real error.

Resources:

Gemini function calling — official playlist: <https://www.youtube.com/watch?v=mVXrdvXplj0>

Groq tool-use docs: <https://console.groq.com/docs/tool-use>

yfinance tutorial (search): "yfinance Python tutorial NSE stocks"

jugaad-data GitHub: <https://github.com/jugaad-py/jugaad-data>

nsepython PyPI: <https://pypi.org/project/nsepython/>

Anthropic's "Building Effective Agents" (provider-agnostic framing, plan's own reference)

ReAct explained (search): "ReAct agent explained" — Krish Naik / CampusX

Week 2 Notes

The ReAct Pattern (Reason → Act → Observe, looped)

The task here is getting the model to answer questions it can't answer from memory alone, by letting it use tools as many times as it needs, in whatever order the question demands. The process is a repeating three-step cycle:

- **Reason** – the model looks at the question and the conversation so far, and decides what's still missing
- **Act** – it calls a tool to go get that missing piece
- **Observe** – it reads whatever the tool returned, and decides whether it now has enough to answer, or needs to reason again and act once more

This cycle repeats — potentially many times — until the model itself decides it has enough to give a final answer. The output is a response that's grounded in real fetched data rather than a single guess made without checking anything.

For example; in TCS-vs-Infosys query: the model reasoned it needed two P/E ratios and a NIFTY option chain, fetched all three in sequence, then reasoned once more over everything it had gathered before writing the final comparison. The learning here is that "agent" really just means "a loop around a model that lets it gather its own evidence before answering," rather than a single question-in, answer-out call. The intelligence isn't in any single step — it's in the model being allowed to repeat the cycle as many times as the question genuinely requires.

Stopping Conditions

The task is preventing the loop from running forever, since nothing about the ReAct cycle itself guarantees the model will eventually stop — a confused or under-specified request could, in principle, have the model calling tools indefinitely. The process uses two independent checks, run after every model turn, in this order:

- **Hard iteration cap first** – has the loop already gone around more times than MAX_ITERATIONS allows? If yes, stop immediately, regardless of what the model wants to do next.
- **Model's own signal second** – if under the cap, check whether the model's last message actually requested a tool. No tool request means it considers itself done.

The first is a forced stop — pure insurance. The second is a natural stop — the model cooperating as intended. The output is that the loop reliably terminates either way, never hanging indefinitely. Both checks are required, not just one: trusting only "the model will stop when it's satisfied" assumes good behavior you can't guarantee, especially once you're chaining unfamiliar tools together; the iteration cap is what protects your API quota and your sanity when that assumption breaks down, independent of how well-behaved the model turns out to be on any given run.

Tool Failure Handling

The task is making sure a failed tool call — a bad ticker, a network hiccup, NSE being temporarily unreachable — doesn't crash your entire script mid-run. The process leans directly on a decision made previously; instead of raising a Python exception when something goes wrong internally, every tool catches its own failures and returns a clean `{"error": "..."}` dictionary instead. In the agent loop, that error dictionary needs no special handling at all — it's passed back to the model as the "observe" step exactly like a successful result would be. From the loop's perspective, there's no difference between a working tool and a failing one; both just produce an observation. The output is that the model actually sees the failure in plain language, in context, and can react to it intelligently:

- try a different tool or a different argument
- tell the user plainly that the data wasn't available
- ask a clarifying question if the failure suggests it misunderstood something

The learning is that good error handling at the tool level is what makes the agent loop simple. You didn't have to write any error-recovery logic in `agent.py` itself - the tools already "speak the model's language" when they fail, so the loop just treats failure and success uniformly.

State Management (StateGraph)

The task is keeping track of everything that's happened across the run; every message, every tool call, every result as the loop goes around multiple times, without anything getting lost, overwritten, or duplicated. The process uses LangGraph's StateGraph, which passes one shared state dictionary through every node in the graph (the agent node, the tools node); each node reads whatever it needs from that state and returns updates, which LangGraph merges back in automatically before handing control to the next node. Two things specifically live in your state:

- **messages** — the full running conversation, growing to include every tool call request and every tool result, not just the user's original question and the final answer
- **iterations** — a simple counter, incremented once per agent turn, used entirely for the stopping-condition check

The output is that by the time the loop ends, the state holds a complete, ordered transcript of the model's entire reasoning process, not merely the final text. This transcript is what makes agents debuggable at all: when something goes wrong or looks off, you can look back through exactly which tool was called, with what arguments, and what came back at each step - instead of only ever seeing a final answer with no visibility into how it got there.

Custom Tool-Execution Nodes (vs. prebuilt ToolNode)

The task is executing whatever tools the model requests in a given turn, while also capturing a detailed, structured log of each call for later reuse.

The process involved a deliberate tradeoff: LangGraph ships a ready-made ToolNode that handles execution automatically with almost no code required, but using `call_tools()` specifically so you could insert your own logging around every individual tool execution - both a `print()` for visibility while running, and an append to `TOOL_CALL_LOG` for a structured, reusable record. The output is a node that behaves identically to the built-in one from the model's point of view, but additionally produces `tool_call_log.json` - a file listing, in order, every tool name called, the exact arguments it was given, and what it returned.

In real engineering tradeoff: convenience frameworks are genuinely great right up until you need visibility they weren't built to give you by default. Writing the more manual version directly pays off - that in future the log becomes the raw material for scoring whether the agent picked the right tools and in the right order, something the prebuilt ToolNode would have given you no easy way to capture.

Graceful Failure on Tool Exceptions and Hallucinated Tool Names

The task is handling two different variety of "something's wrong" without either one crashing the script:

- a **real tool that throws an exception** — e.g. a network error while fetching option chain data
- a **hallucinated tool name** — the model requesting a tool that was never actually given to it, essentially inventing a function that doesn't exist

The process checks these separately, in order. First, for every `tool_call` the model requests, the code checks whether that name actually exists in `TOOL_MAP` at all. If it doesn't, the code never even attempts to call anything — it immediately returns a clean "no tool named X exists" message. If the name is valid, only then does it attempt the call, wrapped in its own `try/except` to catch a genuine runtime failure. Both paths converge into the exact same result shape the model already knows how to interpret from the "observe" step. This ensures that neither failure mode ever takes the script down; both simply become another observation for the model to reason over, same as a successful call would.

Iteration Count ≠ Tool-Call Count

6 tool calls executed successfully despite an iteration cap of 6. This occurs because the iteration counter increments once per agent turn - each time the model reasons - but Gemini can request **multiple tool calls within a single turn**. The `call_tools()` node loops through and executes all requested calls sequentially before returning control to the agent. Consequently, significantly more work fits into an iteration. The cap limits the model's reasoning steps, not total tool usage.

Non-Determinism at temperature=0

The selected content explains why executing the same question twice can result in different tool-usage patterns, such as utilizing a calculation tool in one run and relying on qualitative reasoning in the next. Although setting temperature to 0 minimizes next-word randomness, end-to-end determinism is not guaranteed due to external variables like provider infrastructure batching and timing differences at decision points. While both approaches yielded correct answers, this highlights a key lesson for evaluation design: answer accuracy and tool consistency are distinct metrics that must be evaluated independently, meaning identical tool calls cannot be assumed even at low temperatures.

Retry-with-Backoff on Rate Limits

The task is surviving a 429 RESOURCE_EXHAUSTED error — hitting your free-tier request quota mid-run — without the whole script dying on the spot. The process wraps every single call to the model in a retry loop:

- when a rate-limit error comes back, the code first checks whether the API itself told you exactly how long to wait (a `retryDelay` value, sent back as part of the error response)
- if that value is present, it sleeps for exactly that long before retrying
- if it isn't present, it falls back to a fixed default wait time instead
- this whole cycle repeats up to a set maximum number of attempts before finally giving up and letting a real error surface

The result is an automated script capable of pausing and recovering from temporary quota exhaustion. This issue typically occurs when a single, intensive multi-hop query consumes the majority of your 5-requests-per-minute tier limit prior to the initiation of the subsequent query. Utilizing the API's explicitly recommended wait time is far superior to relying on a predetermined static delay. Doing so ensures you wait only the precise duration required, which is crucial when operating under strict free-tier restrictions where unnecessary idle time should be avoided.

Extracting Plain Text from Structured LLM Response Content

The task is turning the model's final answer into something actually readable. The process exists because answers doesn't come as a simple string the way, sometimes it arrives as a list of structured blocks instead, where each block has:

- a type field (e.g. "text")
- the actual text content itself
- an extras field carrying internal metadata — like a long "signature" blob — that has nothing to do with what a user should ever see

The `extract_text()` helper handles both possible shapes defensively: if the content is already a plain string, it's used as-is; if it's a list of blocks instead, the function pulls out just the text field from each one and joins them together, discarding the metadata. The output is clean, readable final answers printed to your console, instead of a huge unreadable dump of internal signature data mixed into what should be plain text.

Resources:

Tool-Calling Agent in LangGraph: <https://www.youtube.com/watch?v=Nou6Jwsfe7w>

LangGraph Complete Course for Beginners (Vaibhav Mehra):

https://www.youtube.com/watch?v=jGg_1h0qzaM

LangGraph official quickstart docs: <https://langchain-ai.github.io/langgraph/tutorials/introduction/>

ReAct paper (skim): search "ReAct paper explained" for a short video version

Carried-forward open questions (for later weeks, worth keeping in your doc)

Gemini sometimes calls a tool for trivial arithmetic it could answer directly → factor into any future stopping-condition tuning

Raw NSE option chain data has noisy fields (e.g. `impliedVolatility: 0.0` on illiquid strikes) → factor into week 5 evals / data trust assumptions

Tool usage isn't perfectly consistent run-to-run even at `temperature=0` → "correct answer" and "consistent tool usage" need separate scoring in week 5

Week 3 Notes

Part 1 — Data Collection & Chunking

Goal

Turn raw PDFs (company filings/concalls + SEBI regulations) into clean, word-boundary-safe, metadata-tagged text chunks, ready to be embedded. No embedding, no vector DB, no retrieval yet — purely extraction and chunking.

Concepts covered

- **Chunking strategy for messy financial documents** — tables, scanned pages, footnotes all behave differently than clean prose
- **Word-boundary-safe chunking** — cutting a chunk mid-word silently damages retrieval later, since neither half matches a query for the whole term
- **Metadata tagging** — doc_type (filing/regulation) and company (from folder name) let retrieval filter by source type or company later
- **Whitespace collapsing** — raw PDF-extracted text length $\neq$ actual content length once formatting whitespace is normalized; table-heavy documents can lose 40–50%+ of raw length this way
- **Forward-progress and content-gap invariants in a sliding-window loop** — a chunker that doesn't guarantee both can silently hang or silently drop content, with no error thrown either way

Resources

- Google's own reasoning on chunk size/overlap trade-offs (10–20% overlap is standard guidance for prose-heavy text) — general RAG chunking best practice, not tied to one library
- pypdf — used for PdfReader / .extract_text()
- Company IR sources used for real filings:
 - [TCS](#)
 - [Infosys](#)
 - [Reliance](#)
 - [HDFC Bank](#)
 - [ICICI Bank](#)
 - [Tata Motors \(Passenger\)](#)
- [SEBI circulars hub](#)

Key functions (in ingest.py)

Function	Purpose
<code>extract_text(filepath)</code>	Reads a .pdf/.txt file into one text string, page by page for PDFs
<code>chunk_text(text, chunk_size=800, overlap=100)</code>	Sliding-window chunker, word-boundary safe
<code>ingest_folder(...)</code>	Chunks + (in Part 2) embeds one folder's files, tags metadata
<code>ingest_filings(...)</code>	Walks data/, treats every non-"regulations" subfolder as a company
<code>ingest_regulations(...)</code>	Finds the "regulations" folder (any casing) and processes it flat

Note: chromadb/sentence-transformers imports are **lazy** (only loaded inside `main()`), so `extract_text/chunk_text` can be used standalone for testing purpose.

Commands

Smoke test — extraction + chunking on all real documents:

```
cd rag
python -c "
from ingest import extract_text, chunk_text
import glob, os

for filepath in glob.glob('data/*.pdf'):
    text = extract_text(filepath)
    chunks = chunk_text(text)
    company_or_reg = os.path.basename(os.path.dirname(filepath))
    print(f'[{company_or_reg}] {os.path.basename(filepath)}: {len(text)} chars -> {len(chunks)} chunks')
    if len(text) < 200:
        print(f' WARNING: very little text extracted -- possible scanned PDF')
"
```

Edge cases found, and how each was verified

#	Edge case	What happened	Fix	Verification
1	Word cut across chunk boundary	"reflecting" split into "reflect" + "ing" across two chunks	Snap chunk boundaries backward/forward to the nearest space	Manually inspected chunk start/end strings on sample PDF — confirmed clean word boundaries both sides
2	Long space-sparse text (dense numeric tables) causing infinite loop	Process hung/killed (Killed — OOM/timeout) on a synthetic dense-table test string	Added a hard invariant: next_start must always strictly advance past start, guaranteeing loop termination	Re-ran the same pathological input after fix: completed in <0.001s, 2 chunks, no hang
3	Same long space-sparse text silently dropping content between chunks	Forward word-boundary snap could jump past the current chunk's end, skipping the text in between with no error	Capped forward-snap so it can never move past the current chunk's end	Verified via a 2000-word synthetic document — confirmed every sampled word (token0, token137, ... token1999) was present in the reconstructed output
4	Short text (shorter than chunk_size)	Produced 5 overlapping duplicate tail fragments instead of 1 chunk ('short text' → ['short text', 'text', 'ext', 'xt', 't'])	Added an explicit "this chunk already reached the end of the text — stop" check	chunk_text('short text') now correctly returns ['short text']
5	Text exactly equal to chunk_size (800 chars)	Produced 101 near-duplicate tail chunks	Same fix as #4	chunk_text('a'*800) now correctly returns 1 chunk
6	Empty text	—	Explicit early return []	chunk_text("") → []

7	Scanned/image-only PDF pages	Not present in this dataset, but handled	extract_text would return near-empty text for a scanned page; smoke test explicitly flags any file under 200 chars	No files in the actual dataset triggered this warning
8	Whitespace-heavy documents (table-dense fact sheets) inflating raw char count vs. real content	fact-sheet.pdf showed 11,039 raw chars but only 9 chunks — looked anomalous next to similar-sized files	Confirmed <i>not</i> a bug — measured raw vs. whitespace-collapse d length directly	Collapsed length came back at 6,145 chars (55.7% of raw) — 9 chunks is mathematically correct for that much real content
9	Two companies with an identically-named file (e.g. two fact-sheet.pdfs)	Could silently overwrite each other's chunks in the vector store (Part 2 concern, tested now)	Chunk IDs include the company name: filing:{company}:{file name}:{chunk_index }	Verified with two synthetic company folders sharing a filename — confirmed distinct, non-colliding IDs
10	Company folder vs. regulations folder disambiguation	Needed the code to tell "TCS" (a company) apart from "Regulations" (not a company) automatically	Match folder name == "regulations" case-insensitively; treat every other folder as a company	Verified against the exact real folder structure (mixed-case Regulations alongside TCS, Infosys, etc.)

Output of Part 1

17 real documents across 6 companies + SEBI regulations, all cleanly extracted and chunked with correct metadata tagging, zero silent data loss, zero hangs — verified through both synthetic pathological tests and the actual production documents.

Part 2 — Embedding + Hybrid Retrieval

Goal

Take Part 1's clean, chunked text and make it searchable — convert every chunk into a vector embedding, store it in a local vector database (Chroma), and build a retrieval function that combines keyword search (BM25) with semantic search (embeddings), then reranks the results with a cross-encoder for final precision.

Concepts covered

- **Embeddings** — a model (all-MiniLM-L6-v2) converts each text chunk into a vector of numbers capturing its *meaning*. Chunks with similar meaning get similar vectors, even without shared exact words.
- **Vector similarity search** — embed the query the same way, then find stored chunks whose vectors are closest to it. Chroma handles this efficiently.
- **Hybrid retrieval** — combining two different, complementary signals:
 - **BM25** (keyword/lexical) — strong on exact terms: ticker symbols, specific figures, regulation section numbers. Missed paraphrases.
 - **Embedding similarity** (semantic) — strong on paraphrases/concepts (e.g. "profit growth" matching "earnings rose"). Can blur past exact terms.
 - Both scores normalized to 0–1 and averaged.
- **Cross-encoder reranking** — a slower model (cross-encoder/ms-marco-MiniLM-L-6-v2) reads the actual (query, chunk) pair together and re-scores the top ~20 hybrid candidates. More accurate than comparing precomputed vectors, but too slow to run over the whole corpus — hence "narrow cheaply, then rerank the shortlist," the standard production pattern.
- **Metadata filtering** — Part 1's doc_type/company tags let a search restrict to one document type or one company without rescanning everything.
- **Unbounded relevance scores** — the cross-encoder's output is a raw real number, not a 0–1 probability. Negative scores are normal; what matters is the relative gap between queries, not an absolute "good/bad" cutoff.

Resources

- [ChromaDB's cross-encoder reranking cookbook](#)
- [sentence-transformers](#)
- [rank_bm25](#)
- [Hybrid search + reranking reference](#)

Files

File	Purpose
ingest.py	Embeds every chunk (all-MiniLM-L6-v2) and stores it in Chroma (main())
retrieval.py	Pure hybrid search logic — hybrid_search(query, doc_type, company=None). No tool-wrapping (that's Part 3).
test_retrieval.py	Manual test script — runs sample queries, prints results

Commands run, in order

Build the vector store: python ingest.py

First run downloads the embedding model (~91MB) from Hugging Face — one-time, cached afterward.

Test retrieval: python test_retrieval.py

First run downloads the reranker model (~91MB), also one-time/cached.

Output (summarized, 4 test queries):

Query	Result
"revenue growth this quarter" (no filter)	Real revenue/growth commentary from TCS & Infosys, correct QoQ/YoY figures, scores +3.5 to +6.2
"earnings this quarter", company="TCS"	All 5 results genuinely TCS-only — no cross-company leakage
"lot size margin expiry day" (regulations)	All 5 results from actual regulation circulars — zero filing chunks mixed in
"weather forecast for Mumbai tomorrow" (nonsense, sanity check)	Scores -6.8 to -8.1 — sharply lower than even the legitimate regulation query's -0.2 to -2.1, correctly signaling irrelevance

Edge cases found, and how each was verified

#	Edge case	What happened	Fix	Verification
1	Dead code in ticker filtering	An earlier draft computed a <code>where_filter</code> for Chroma that was built but never actually passed into the query — real filtering silently happened via crude filename substring matching instead	Removed the dead variable; filter directly against the company metadata field (set from folder names in Part 1)	Confirmed "reliance" correctly matches folder/company "Reliance Industries" without depending on filename content
2	Import-time dependency coupling	Originally chromadb/sentence-transformers were imported at module level, meaning just importing the file required both installed	Moved imports inside <code>_get_resources()</code> (lazy-loaded, only on first real search)	Verified via a simulated environment with both packages blocked — module still imports cleanly
3	Company filter matching nothing	Untested previously — could plausibly error instead of returning empty	Post-filter step naturally returns <code>{}</code> → <code>[]</code> if no metadata matches	Verified with a nonexistent company name — returns <code>[]</code> , no exception
4	Re-running ingestion on the same data	Could plausibly duplicate every chunk on a second run	Chroma upsert (not add) — overwrites by ID rather than appending	Ran <code>ingest.py</code> 's pipeline twice against identical input — chunk count stayed the same both times
5	<code>doc_type</code> cross-contamination (filing vs. regulation)	Untested against real data previously — only verified on synthetic samples	<code>where={"doc_type": doc_type}</code> scopes both the BM25 index and the vector query	Confirmed on real data: filing and regulation queries returned zero overlapping sources

6	Interpreting negative relevance scores	Could look like retrieval "failed" at a glance	N/A — this is expected model behavior, not a bug	Explained via relative comparison: real query scores (+3 to +6) vs. nonsense query scores (-6.8 to -8.1) — a ~10+ point gap confirms the reranker is discriminating correctly
7	No relevance floor	hybrid_search always returns top-N results regardless of how weak the scores are — even a totally unrelated query still gets 5 results back	Not yet fixed — open design question carried into Part 3: whether to add a minimum-score threshold so the tool can return "no relevant results" instead of forcing through weak matches	Directly observed: the nonsense weather query still returned 5 chunks, just with clearly bad scores

Output of Part 2

A local Chroma vector store (chroma_db/, gitignored) containing all 813 real chunks from 6 companies + SEBI regulations — embedded, hybrid-searchable (BM25 + semantic), and rerank-verified against real queries on your actual corpus. Company filtering, doc_type isolation, and re-ingestion safety all confirmed on production data, not just synthetic tests.

Part 3 — Wrap as Tools + Wire into the Agent

Goal

Take *hybrid_search()* and make it something the agent can actually reach for - wrap it as two LangChain tools, add them to the agent's tool list, and prove the agent can autonomously choose between live market data, document search, and regulation search across a single multi-hop question.

Concepts covered

- **Thin tool adapters** - *retrieval_tools.py* contains zero retrieval logic of its own; it wraps *retrieval.py*'s *hybrid_search()* in `@tool`-decorated functions, the same pattern as *agent_loop/langgraph_tools.py* wrapping *tool_calling/tools_def.py*. Every fix made (relevance threshold, company filtering) is inherited automatically — nothing duplicated to keep in sync.
- **Tool list composition** — `ALL_TOOLS = MARKET_TOOLS + RAG_TOOLS` is just list concatenation. The model sees 7 tools with no built-in distinction between "live data" and "document search" — it picks based on each tool's description alone.
- **Cross-domain multi-hop reasoning** — the real test wasn't just "can it call two tools," it was "can it correctly split one question into two different kinds of sub-questions" (a document search and a regulation search), the harder version of what Week 2 proved with same-domain tools.
- **Daily vs. per-minute rate limits** — these are two genuinely different failure modes requiring different handling. A per-minute limit (RPM) is recoverable with a short wait; a daily limit (RPD) is not recoverable until a ~24h reset, so retrying it is actively wasteful, not just unhelpful.
- **Iteration cap vs. task success** — hitting a safety cap mid-reasoning doesn't mean the agent failed; it means the loop needs a graceful exit path so a real answer still reaches the user instead of a blank response.

Files

File	Purpose
<code>rag/retrieval_tools.py</code>	Wraps <code>hybrid_search()</code> as <code>search_filings(query, company=None)</code> and <code>search_regulations(query)</code>
<code>agent_loop/agent.py</code>	Updated — imports <code>RAG_TOOLS</code> , combines with the 5 existing market tools, adds cap-recovery fallback and daily-quota handling

Output (final run, summarized) (*python agent.py*)

- **Queries 1–3** (Week 2 tools): identical quality to prior validated runs — no regression from adding 2 more tools to the list.
- **Query 4** (Week 3 target): called `search_filings` (Reliance) and `search_regulations` across 6 tool calls, hit the 6-iteration safety cap mid-reasoning, and the cap-recovery fallback produced a complete, correctly-sourced final answer — Jio subscriber numbers attributed to the filing, SEBI's Nov 2024 contract-size circular attributed to the regulation search, explicitly noting the data was Q1 FY27 (not the Q3 asked for) rather than pretending otherwise.

Edge cases found, and how each was verified

#	Edge case	What happened	Fix	Verification
1	Duplicated pre-fix retrieval logic	An earlier draft of <code>retrieval_tools.py</code> had its own full copy of hybrid search, including Part 2's already-fixed dead-code ticker filter bug	Rewritten as a thin wrapper over <code>retrieval.py</code> — one implementation, not two	Verified via mocked test harness: all 7 tools register correctly on the combined list
2	Redundant re-querying	The model repeatedly called <code>search_filings</code> with only minor rewording (5–6 times) chasing an exact "Q3" label that doesn't exist in the corpus (only Q1 FY27 was ingested)	Added explicit system-prompt instruction: use the best available data and disclose the period mismatch, rather than re-searching for an exact match	Confirmed on real data: tool-call count dropped from 6 redundant filing searches to a mix of both tools; final answer explicitly disclosed the Q1-vs-Q3 discrepancy instead of hallucinating or looping forever

3	Iteration cap hit mid-tool-call → silent empty answer	First real run returned [final answer] [] — the cap forced an "end" state while the model still had a pending, unexecuted tool request	Added a cap-recovery fallback: one final call with tools disabled, forcing a text answer from whatever was already gathered	Verified two ways: (a) simulated the exact cutoff state from the real failure and confirmed it now produces a real answer, (b) confirmed on the actual second real run — hit the same cap, recovered cleanly with a full answer
4	Daily quota vs. per-minute rate limit treated identically	gemini-3.5-flash hit a daily cap (20 requests/day on this key) but the existing retry logic treated it like a transient per-minute limit — retried 3 times over ~97 seconds before crashing anyway	Added detection for PerDay/RequestsPerDay in the error message; fails immediately with a clear explanation instead of wasting retries on an unrecoverable-within-the-day error	Simulated the exact error payload from the real crash — confirmed it now fails in one attempt, 0 wasted time, with a message distinguishing it from the recoverable case
5	Script-ending crash on any single query's permanent failure	An uncaught daily-quota exception took down the whole script, losing the tool-call log from queries that had already succeeded	Wrapped the query loop in try/except for the new DailyQuotaExhausted exception; stops cleanly, reports how many queries completed, still saves the log	Verified via simulation: 1 of 3 queries succeeds, 2nd hits quota, script reports Completed 1/3 and exits cleanly rather than an ugly traceback

Week 3 overall

An agent with 7 tools spanning live market data, company filings/concalls, and SEBI regulations — verified end-to-end on real ingested data (813 chunks) and real Gemini API calls, correctly splitting a cross-domain question into the right sub-searches, recovering gracefully from both a reasoning-loop cap and two distinct categories of API rate limiting. This closes Week 3 per the original 6-week plan.

Week 4 Notes

Part 1 — Conversation Buffer Memory

Goal

Turn the agent from a one-shot question-answering script into something that can actually hold a conversation — remembering what was said earlier in the same session, so follow-up questions using "that," "it," "which one," etc. resolve correctly without the user re-stating context.

Concepts

- **Checkpoint** — LangGraph's mechanism for saving a snapshot of graph state after every step. Without one, every `.invoke()` call starts from a completely blank state, no matter how many times you've called it before.
- **thread_id scoping** — the checkpointer doesn't just remember "the conversation," it remembers *a specific conversation*, keyed by `thread_id`. Same `thread_id` across calls = shared history. Different `thread_id` = a totally separate, isolated conversation. This is what lets one running agent serve multiple independent conversations without them bleeding into each other.
- **Session vs. persistent memory** — `InMemorySaver` (used here) keeps state in RAM only; it's gone the moment the script exits. This is deliberate for upgrades to `SqliteSaver`, which writes to disk and survives between separate runs.
- **Reducer vs. plain state fields** — `messages` is an Annotated field with an `add_messages` reducer, so each new turn's messages get *appended* to history automatically. `iterations` is a plain field with no reducer, so it *replaces* rather than accumulates — a distinction that turned out to matter a lot (see the bug below).

Resources

- LangChain's official memory docs (current, matches installed LangGraph version): <https://docs.langchain.com/oss/python/langgraph/add-memory>
- LangGraph persistence deep-dive (checkpointer types, `MemorySaver` → `SqliteSaver` → `PostgresSaver` progression): referenced from search, covers exactly the Dev → Local → Production path this project follows across Parts 1 and 2

Commands

```
cd agent_loop
python agent.py
```

Launches the interactive loop directly — no flags needed for normal use.

Output

Turn	Question	What happened
1	"What is TCS's P/E ratio?"	1 tool call (get_company_overview), answered 17.29
2	"How does that compare to Infosys?"	Correctly resolved "that" → TCS's P/E. Only called the tool for Infosys — reused TCS's already-known P/E from memory instead of re-fetching it
3	"Which one would you say is cheaper right now?"	Correctly resolved "which one" across both prior turns. Recognized "cheaper" was ambiguous (valuation vs. price), fetched <i>only</i> the new data needed (stock prices), and reused both companies' already-known P/E ratios rather than re-fetching anything

The memory reuse in turns 2–3 wasn't explicitly coded — it fell out naturally from giving the model access to its own conversation history via the checkpoint.

Edge cases found, and how each was verified

#	Edge case	What happened	Fix	Verification
1	Iteration cap scope	If iterations isn't explicitly reset each turn, it would persist and accumulate across the <i>entire</i> conversation (since checkpointed state carries forward), meaning the 6-call safety cap would apply cumulatively to the whole session instead of per-question — a long conversation would hit the cap after just a few total turns even if each one only needed 1-2 tool calls	Explicitly pass iterations: 0 in the input to every .invoke() call. Since iterations has no reducer, this input value <i>replaces</i> the checkpointed value rather than adding to it — while messages (which does have a reducer) correctly still accumulates	Directly tested with a mocked LLM across 3 same-thread turns: confirmed iterations stayed at 1 each turn (not 1→2→3) while messages correctly grew 2→4→6

2	Thread isolation	Untested previously — needed to confirm two different conversations don't accidentally share history	N/A — this is thread_id's job by design	Verified: a second, different thread_id showed only 2 messages (its own), with zero bleed-through from a first thread that had 6
3	Garbled/duplicated user input	Real run: terminal artifact caused "Which one would you say is cheaper right now?" to be sent twice concatenated in one message	Not a code bug — no fix applied	Confirmed the model handled the malformed input gracefully and answered correctly anyway; noted as a terminal quirk, not a script issue
4	Daily quota during interactive use	A chat() session could hit the daily cap mid-conversation same as regression_test() could	Already covered by the DailyQuotaExhausted handling from Week 3 Part 3 — chat() catches it and exits the loop cleanly instead of crashing	Reused existing, already-verified exception handling; no new test needed since the underlying mechanism is unchanged

Output of Part 1

An interactive, multi-turn agent where conversation history persists for the duration of a session — confirmed on real data with real multi-hop pronoun resolution and genuine memory-driven tool-call avoidance (not just context retention, but the model actively choosing *not* to re-fetch data it already has). Session-scoped only — closing the terminal loses everything.

Part 2 — Persisted Watchlist

Goal

Give the agent a way to remember "what the user is tracking" that survives *between* sessions — not just within one conversation, but across closing and reopening the terminal entirely.

Concepts

- **Separating conversation memory from application state** — InMemorySaver remembers *what was said*; SQLite watchlist remembers *a fact the user established*, independent of any particular conversation. These are different kinds of memory with different lifetimes, which is why they're two separate storage mechanisms.
- **Upsert semantics** — re-tracking an already-tracked symbol updates it in place rather than creating a duplicate row, keyed on symbol as the primary key.
- **Mixed-type reference resolution** — the hard part isn't storing two instrument types, it's a *later* question correctly routing different follow-ups to different types without the user re-specifying which is which.
- **Tool-call ordering via prompt instruction** — explicitly telling the model to call `view_watchlist` *before* answering watchlist-referencing questions, rather than assuming it would infer this on its own.

Files

File	Purpose
agent_loop/watchlist.py	Plain SQLite CRUD — <code>add_instrument</code> , <code>remove_instrument</code> , <code>list_watchlist</code> . No LangChain.
agent_loop/watchlist_tools.py	Thin <code>@tool</code> wrappers: <code>add_to_watchlist</code> , <code>remove_from_watchlist</code> , <code>view_watchlist</code>
agent_loop/agent.py	Updated — 10 tools total now (5 market + 2 RAG + 3 watchlist); system prompt extended with explicit watchlist-first, type-aware-routing instructions

Commands

```
python agent.py
```

Output

Turn 1 — "Track HDFC Bank, ICICI Bank, and NIFTY weekly options": 3 correct add_to_watchlist calls, correct instrument_type for each (2× equity, 1× option), clean confirmation.

Turn 2 — "How are their margins and OI trends looking?":

1. Called view_watchlist first, exactly as instructed
2. Routed get_option_chain → NIFTY only, producing a real PCR calculation (0.96, independently verified correct against the raw OI numbers) plus support/resistance levels
3. Routed get_stock_price + get_company_overview + search_filings (NIM-specific query) → both banks, correctly interpreting "margins" as Net Interest Margin for banks specifically rather than confusing it with options margin
4. Final answer organized per-instrument, each with domain-appropriate metrics — no blending, no misapplied terminology

Edge cases found, and how each was verified

#	Edge case	What happened	Fix	Verification
1	Duplicate symbol re-tracked	Could plausibly create a second row for the same instrument	symbol set as PRIMARY KEY with ON CONFLICT ... DO UPDATE (upsert)	Re-added HDFCBANK.NS with a new label — confirmed row count stayed the same (updated, not duplicated)
2	Case sensitivity	'hdfcbank.ns' vs 'HDFCBANK.NS' could be treated as two different symbols	Symbols normalized to uppercase before every read/write	Re-added lowercase version — confirmed it updated the existing uppercase row rather than creating a new one
3	Invalid instrument type	e.g. 'crypto' — should fail cleanly, not corrupt the table	Explicit validation against ('equity', 'option') before touching the DB	Confirmed clean {"error": ...} response, no partial write

4	Removing a never-tracked symbol	Could plausibly throw	Checks cursor.rowcount, returns a graceful "not_found" status	Confirmed no exception, clean status message
5	Empty/whitespace-only symbol	Could silently insert a junk row	Explicit empty-check after stripping whitespace	Confirmed rejected with a clean error
6	Cross-process persistence (the actual core requirement)	Untested until directly verified	N/A — inherent to using a real .db file rather than in-memory storage	Added items in one Python process, started a completely separate process , confirmed it read back the exact same watchlist
7	Model correctly applying type-specific vocabulary ("margin" meaning NIM for banks vs. margin for options)	Not something explicitly coded — a risk the model could conflate the two "margin" meanings	N/A — emerged from the system prompt's general instruction to route appropriately by type, not a hardcoded rule for this specific term	Confirmed on the real run: retrieved actual NIM percentages from filings for the banks, actual OI/PCR data for NIFTY — zero cross-contamination between the two concepts

Output of Part 2

A watchlist that genuinely persists across sessions (verified via separate-process testing, not just code inspection), correctly mixing equity and F&O instrument types, with the agent demonstrating real domain-aware reference resolution — not just mechanical type-routing, but correctly interpreting domain-specific terminology (margin) differently depending on instrument type, and backing every figure with real retrieved data rather than guesses.

Part 3 — Summarization + Long-Conversation Stress Test

Goal

Prevent the conversation buffer from growing unbounded - compress older turns into a running summary once the message count crosses a threshold, and prove reference resolution still works correctly after that compression happens, not just before it.

Concepts

- **Context window management** — every raw message costs tokens on every subsequent call; a long conversation left uncompressed eventually becomes expensive and risks hitting the model's context limit.
- **The standard LangGraph summarization pattern** — RemoveMessage actually deletes messages from checkpointed state (not just hides them from one call), paired with a running summary string that gets prepended as context so compression doesn't mean forgetting.
- **Reducer vs. plain field, applied a second time** — messages (reducer field) supports deletion via RemoveMessage; summary (plain field) just gets overwritten each time, same mechanic as iterations from Part 1.
- **Sliding window + summary, not full compression** — the most recent messages stay verbatim (sharp, exact context for the current exchange), only older messages get compressed, balancing cost against fidelity.
- **Grounding vs. inference** — the difference between an agent that answers from conversation memory (fast, sometimes right) and one that verifies against actual persisted state (always correct) — this mattered concretely in one of the test findings below.

Files

File	Change
agent_loop/agent.py	Added summary field to AgentState; call_agent prepends it as context; new summarize_if_needed() function; ask() calls it after every turn; new stress_test() (--stress-test flag); MODEL pulled into a named constant

Commands run

```
cd agent_loop
```

```
python agent.py --stress-test
```

Edge cases found, and how each was verified

#	Bug	What happened	Fix	Verification
1	Summarization call sent only a SystemMessage	Crashed with ValueError: contents are required — Gemini's API requires actual conversation content separate from system instructions; a request with only a system message has no real contents	Changed the summarization request to a HumanMessage instead	Confirmed via mocked test that captured the actual message type sent — HumanMessage, matching what Gemini requires
2	15-turn stress test itself was oversized for the actual test need	Burning quota on turns that didn't test anything new — summarization was already confirmed triggering by turn 6	Trimmed stress_test() to 8 turns: 5 to build context + reliably trigger summarization, 3 focused post-summarization checks	Directly confirmed on the real (shortened) run — summarization still triggered as expected
3	Final "summarize the conversation" turn skipped view_watchlist entirely	Answered from conversation memory alone, initially looked like a hallucination (claiming TCS was tracked)	Strengthened the system prompt: explicitly require view_watchlist before ANY watchlist-referencing answer, including conversation summaries; explicitly warn against conflating "discussed" with "tracked"	Re-tested directly — view_watchlist now fires correctly before answering

Output of Part 3

A conversation memory system that compresses rather than truncates — proven across two independent summarization cycles in one real run, with reference resolution surviving compression correctly for both equity and F&O instrument types. Along the way, caught and fixed a real API-format bug, navigated two rounds of model/quota instability by switching from guessing at model names to querying the account directly, and caught a grounding gap in how the agent answered conversation-summary questions.

Week 5 Notes

Week 5, Part 1 — The Eval Set

Goal

Move towards a scored, reproducible eval process. Part 1's specific job: a held-out eval set covering all five tool types plus multi-hop combinations, with enough structure that Parts 2 and 3 can score against it programmatically rather than by eyeballing transcripts.

Concepts covered

- **Held-out eval sets** — a fixed, known set of test questions written before running them, so scoring isn't influenced by having already seen how the agent responds. Written once, reused across every future eval run.
- **Ground truth vs. behavioral correctness** — two different things being tested here. Live-data questions (price, fundamentals, option chain) don't have a fixed "correct answer" since real market data changes daily — what's checkable is behavior (right tool called, no error, plausible output). Filing and regulation questions are grounded in static, already-ingested PDF snapshots, so they do have a fixed, checkable ground_truth.
- **Deliberately adversarial test cases** — several questions are designed to fail if the agent isn't careful: an ambiguous ticker (Tata Motors' corporate split), a genuinely unanswerable question (gold prices — no tool covers commodities), and critically, the Q3/Q1 period-mismatch trap.
- **Multi-hop question design** — questions require 2-3 different kinds of tools in one ask, escalating in complexity up to a 3-tool-type question, to stress-test tool selection under compound requests.

Files

evals/eval_set.py → The 33-question eval set as a list of dicts, plus a summary() validation function

Location: evals/, at project root — sibling to agent_loop/, rag/, tool_calling/. Deliberately its own folder since the file has zero dependency on agent_loop/'s code (no imports beyond the standard library) — it's data, not agent logic, and this is also where Part 2/3's scoring code will live alongside it.

Structure of each question

```
python
{ "id": "filing_05",
  "category": "filing",
  "question": "...",
  "expected_tools": ["search_filings"],
  "ground_truth": "...", # fixed fact, or None for live-data questions
  "notes": "...",      # what this question is specifically checking for }
```

Design decisions and what each question is actually testing

Category	Count	Notable test cases
Price	5	price_05 — commodity question with no matching tool, checks graceful decline over hallucination
Fundamentals	5	fund_05 — Tata Motors' ambiguous post-split ticker, checks the agent flags ambiguity rather than silently guessing
Option chain/OI	5	opt_03 — checks PCR is actually computed via calculate, not eyeballed
Filing	6	filing_05 — the single most important check in the set: re-tests the exact Q3-asked/Q1-available honesty requirement first fixed in week 3
Regulation	6	reg_04 — the oldest, most topically-different circular (commodities, 2019) in the corpus, checks retrieval doesn't just default to the newest/largest document
Multi-hop	6	multi_02 — the week 3 target query, reused verbatim as a regression check; multi_06 — deliberately messy 3-tool-type question, where a reasonable answer might legitimately push back on a flawed premise rather than force an answer

Zero API calls made ⇒ This entire part cost nothing — pure data/design work, verified with local validation only.

Output of Part 1

A validated, structurally sound 33-question eval set — balanced across all 5 tool types, JSON-serializable for downstream tooling, with 12 questions carrying fixed ground truth for RAGAS scoring and several deliberately adversarial cases (ambiguous tickers, unanswerable questions, the period-mismatch trap) built in from real bugs this project already found and fixed.

Week 5, Part 2 — RAGAS Metrics on the Filings and Regulations Corpora

Goal

Score retrieval quality (not full agent generation) using RAGAS's context precision and context recall metrics, run separately against the filings corpus and the SEBI regulations corpus.

Concepts covered

- **Context Precision** - of the chunks retrieval actually returned, how many were genuinely useful? (signal-to-noise on the retrieval side)
- **Context Recall** - everything actually needed to answer correctly, how much did retrieval actually find? (completeness)
- **Reference-based metrics without generation** - both metrics only need question + retrieved_contexts + ground_truth, not a live-generated answer. This kept API usage to ~2 judge calls per question instead of 4+, since no separate "generate an answer" step was needed.
- **LLM-as-judge evaluation** - RAGAS uses an LLM (Gemini here, via LangchainLLMWrapper) to judge whether retrieved chunks are relevant/sufficient, rather than exact-string matching - which is both more flexible and more failure-prone in ways that surfaced directly in this run.
- **Concurrency vs. rate limits** - RAGAS defaults to 16 concurrent judge calls; free-tier APIs can't sustain that. Sequential execution (max_workers=1) trades wall-clock time for reliability.
- **Resumable batch scoring** — treating an eval run as checkpointable state (via a results log) rather than an all-or-nothing script, so a partial failure doesn't waste already-completed work.

Files

File	Purpose
evals/ragas_eval.py	Main scoring script — builds a RAGAS dataset from eval_set.py + rag/retrieval.py, scores it, appends results
evals/ragas_results.jsonl	Accumulated results — one line per question per run, used for resumability and reused by Part 3
evals/diagnose_retrieval.py, evals/diagnose_filing_04.py	Zero-cost diagnostics — inspect what hybrid_search() actually returns for a specific failing question, no API calls

Commands run

cd evals

python ragas_eval.py --force --limit 3 --key 0 # Part A: filing_01, filing_02, filing_03

python ragas_eval.py --limit 3 --key 1 # Part B: filing_04, reg_04, reg_05

python diagnose_retrieval.py # free diagnostics, no API cost

python diagnose_filing_04.py

Final results — all 9 scorable questions

Question	Precision	Recall
filing_01 (Jio subscribers)	0.500	0.000
filing_02 (HDFC NIM)	1.000	1.000
filing_03 (ICICI NIM)	0.000	0.000
filing_04 (TCS workforce)	0.000	0.000
reg_01 (contract size)	0.833	1.000
reg_02 (option premium date)	0.583	1.000
reg_03 (calendar spread)	0.679	1.000
reg_04 (uniform lot size)	0.750	1.000
reg_05 (weekly index)	1.000	0.000

Category averages:

Filings — 0.375 precision / 0.250 recall. Regulations — 0.769 precision / 0.800 recall.

Edge cases and bugs found, and how each was resolved

#	Issue	What happened	Fix	Verification
1	RAGAS import completely broken	Hardcoded import of ChatVertexAI from a langchain_community submodule that no longer exists in current releases — a confirmed, currently-open upstream bug, not a local setup mistake	Compatibility shim: inject a harmless stub module into sys.modules before importing ragas, satisfying the broken import without needing the real (unused) VertexAI package	Confirmed a version-downgrade "fix" instead breaks langchain-core compatibility with langgraph/langchain-google-genai; the shim avoids that trade-off entirely — verified ragas imports cleanly with it
2	Deprecated metrics import path	ragas.metrics.ContextPrecision/ContextRecall prints deprecation warnings pointing to ragas.metrics.collections	Deliberately kept the deprecated (but stable, well-documented) path — the newer path requires a different, less-documented constructor (InstructorBaseRagasLLM) and looked actively in flux	Verified both import paths work; chose stability over avoiding a warning
3	Massive concurrent rate-limit failures	First real run: RAGAS's default max_workers=16 fired 16 concurrent judge calls, overwhelming the free-tier per-minute limit — most jobs timed out, results were mostly NaN	RunConfig(max_workers=1, timeout=180) — fully sequential judge calls	Confirmed on a second real run: zero timeouts across 6 sequential calls, all real scores

4	Wasting quota re-scoring already-succeeded questions	A key hitting its daily cap mid-run meant losing all progress on retry, including questions that had already scored successfully	<code>_load_already_scored_ids()</code> — treats a question as done only if BOTH metrics are real (non-NaN); skips it on subsequent runs unless <code>--force</code>	Verified with a simulated results file matching the actual failure pattern — correctly skipped 3 already-done questions, correctly re-attempted 2 partially-failed ones
5	Non-deterministic chunking	Needed to split 9 questions into small, quota-safe batches ("Part A/B/C") that are reproducible run to run	<code>--limit N</code> iterates <code>EVAL_SET</code> in its own fixed list order	Verified <code>--force --limit 3</code> always yields the same 3 questions (filing_01, 02, 03) run to run
6	filing_01 scored recall=0 despite the right chunk being retrieved	Diagnostic showed the chunk did say "Jio total subscriber base of over 533 million" — a real answer — but <code>ground_truth</code> specified "533.3 million," and RAGAS's judge apparently didn't credit the looser phrasing as sufficient	Not fixed — logged as an eval-authoring lesson (ground truth phrasing affects scoring strictness), relevant to Part 3's failure-mode review	Confirmed via free local diagnostic (no API cost) — this is a scoring-calibration issue, not a retrieval bug
7	filing_03/filing_04 scored 0/0 — genuine retrieval misses	Correct company retrieved both times, but the specific answer-bearing chunk (NIM table, workforce figure) never surfaced in the top results	Not fixed yet — filing_04's diagnostic returned only 3 results instead of the usual 5, suggesting <code>retrieval.py</code> 's <code>MIN_RELEVANCE_SCORE</code> floor may be filtering out a real answer .	Confirmed via free local diagnostics for both questions

Output of Part 2

A complete, quantitatively-scored retrieval quality baseline across both corpora — filings (0.375/0.250) meaningfully weaker than regulations (0.769/0.800), matching a qualitative observation made back in week 3 with real numbers behind it now. Three distinct, diagnosed failure patterns identified (strict ground-truth phrasing, genuine retrieval misses, and a possible relevance-floor trade-off).

Week 5, Part 3 – Tool-Call Correctness + Failure Analysis

Status: COMPLETE. 33/33 questions scored.

Score whether the agent calls the right tools on each of the 33 eval questions (separate from Part 2's RAGAS retrieval-quality scoring, which only covers filing/regulation categories), then identify and fix the worst failure modes found along the way. Broken into 5 completed segments — a scorer, a live-run harness, two live batches, and an aggregate report — run in that order because each segment's output is a required input for the next one.

What actually happened diverged from the original plan: the "identify & fix the worst 3 failure modes" step ended up happening continuously during the live batches instead, because the first batch immediately surfaced a real bug (Segment 3) that then reproduced again in the second batch (Segment 4), motivating three more rounds of fixes before the aggregate report was even built. By the time Segment 5 ran, 4 bugs had already been found, fixed, and live-verified — ahead of schedule, but in a more improvised order than planned.

Segment 1 — The Scorer

Goal

Score how well the agent's actual tool calls match `eval_set.py`'s `expected_tools`, entirely offline.

Concepts covered

- **Multiset (Counter) comparison, not set membership:** `fund_02` expects `get_company_overview` called TWICE (once per ticker being compared). A set-based check would treat one call as a full pass; Counter intersection scores duplicate expectations correctly.
- **Recall and precision kept separate, never collapsed to one number:** They catch genuinely different failure modes: low recall means a call was missed; low precision means an unnecessary or wrong call was made. Every later finding depended on being able to tell these apart.
- **Vacuous-case handling for the one "expect nothing" question:** `price_05` (the gold-price decline case) expects zero tool calls. `score_tool_calls()` returns `recall=1.0` when nothing was expected and nothing was missed, but `precision=0.0` if a tool WAS called anyway - this is exactly what later caught `price_05`'s GOLDBEES.NS substitution as a precision hit worth reviewing, rather than silently passing or failing it.
- **Working around a real gap in agent.py's existing logging:** `TOOL_CALL_LOG` is cleared at the start of every `ask()` call, and `chat()/regression_test()/stress_test()` only ever write it to disk once, at the very end of a whole run — so the pre-existing on-disk log only ever captured the LAST question's calls, useless for scoring 33 separate questions. `capture_tool_calls_for_question()` reads `TOOL_CALL_LOG` immediately after each individual `ask()` call, before the next call clears it, giving one clean, correctly-scoped record per question.

Files

`evals/tool_eval.py` → `score_tool_calls()`, `capture_tool_calls_for_question()`, 7 self-tests

Commands

python tool_eval.py

Output

All 7 self-test cases passed.

Coverage check: 32/33 questions have non-empty expected_tools (price_05 is the one deliberate exception). Zero API calls made.

Edge cases covered by the self-tests

#	Case	What it checks	Result
1	Perfect single match	Baseline correctness	recall=1.0, precision=1.0
2	Duplicate expected, both made	fund_02-shaped questions score correctly	recall=1.0, precision=1.0
3	Missing one of two expected	A real miss doesn't get masked	recall=0.5, precision=1.0, missing=["get_company_overview"]
4	Correct call + one unnecessary extra	Extra work penalizes precision only, not recall	recall=1.0, precision=0.5
5	Correctly declined	Zero expected, zero called	recall=1.0, precision=1.0
6	Hallucinated call when none expected	Precision correctly flags it	recall=1.0, precision=0.0
7	Wrong tool entirely	Total miss scored as total miss	recall=0.0, precision=0.0

A validated, zero-cost scorer, plus a harness function built but not yet exercised live — ready for Segment 2 to wrap in an actual batch runner.

Segment 2 — Live-Run Harness

Goal

Build the script that runs eval questions through agent.py live and scores them, resumably, so a quota cutoff mid-batch (expected, not exceptional) never loses already-completed work or requires manual bookkeeping.

Concepts covered

- **Isolated thread_id per question, never a shared conversation:** Sharing a thread (the way chat() does) would let the model dodge a real tool call by pulling data from conversation memory left over from an earlier question this would understate recall failures, not just be slightly unfair.
- **Resumability via error-tagged JSONL:** checked at load time, not write time. A question counts as "already scored" only if its stored line has no error field. A question that errors out mid-batch — whether from an unrelated exception or a quota cutoff — is NOT marked done, so the next run retries it automatically.
- **Zero-cost list preview:** added specifically because the harness was built before the real eval_set.py's exact category strings were known — verified against the real file before any live run, catching a filter mistake for free instead of on a live quota-costing run.
- **Automatic key rotation:** agent.py's QUESTIONS_PER_KEY counter is module-level state; because Python caches the agent import, calling the harness's capture function repeatedly in a loop lets rotation keep working across a whole batch with zero extra harness code.

Files

File	Purpose
evals/run_tool_eval.py	Batch runner: --ids, --category, --limit, --force, --sleep, --list
evals/tool_eval_results.jsonl	Accumulated results, one line per question run, resumable

Commands

```
python run_tool_eval.py --list # zero-cost preview
python run_tool_eval.py --category price,fundamentals,option_chain --limit 3
python run_tool_eval.py --category filing,regulation
python run_tool_eval.py --category multi_hop --ids multi_03 --force
```

Output

Verified against a deterministic mocked agent.py before any real quota was spent: resumable skip on re-run confirmed, force override confirmed, DailyQuotaExhausted confirmed to stop the whole batch cleanly with no stray result written, a generic exception confirmed to log and let the batch continue, and list's output confirmed to match the real eval_set.py's actual category strings once shared (price, fundamentals, option_chain, filing, regulation, multi_hop).

Edge cases found, and how each was verified

#	Edge case	What happened	Fix / behavior	Verification
1	Re-running a completed batch	Should skip everything already scored	<code>_load_already_scored_ids()</code> filters on absence of error	Mocked 2nd identical run: 0 questions processed
2	Forcing a re-run	<code>--force</code> needed to bypass the skip for named ids only	<code>_select_questions()</code> bypasses the skip filter when <code>force=True</code>	Mocked: <code>--ids price_01 --force</code> re-ran despite already being scored
3	Quota dies mid-batch	Must not mark the interrupted question done; must stop the batch, not just skip one question	<code>DailyQuotaExhausted</code> caught at the batch-loop level, breaks immediately, no result appended	Mocked <code>TRIGGER_QUOTA</code> question: batch stopped, no result line written, next question never attempted
4	Unrelated exception on one question	Should not take down the rest of the batch	Caught separately, logged with error set, loop continues	Mocked <code>TRIGGER_ERROR</code> question: logged, batch proceeded to the next question in the same run
5	Category filter vs. the real file	<code>--list</code> needed to reflect actual <code>eval_set.py</code> strings, built before that file was shared	Filter logic fully dynamic, no hardcoded category names	Confirmed exact match against the real file, including <code>price_05</code> 's empty <code>expected_tools</code>

A resumable, quota-aware batch runner, confirmed correct against both a mock and the real eval set, ready for live use.

Segment 3 — First Live Batch (price, fundamentals, option_chain)

Goal

Run the three categories Part 2's RAGAS scoring never touched (no retrieval component to score there), producing the first tool-call data on them at all.

All 15 questions, across 3 separate runs.

Final state: 15/15 scored.

Output

Recall 1.000 across all 15. Precision losses split into two kinds:

- Good-faith "extra rigor": price_05 (GOLDBEES.NS substituted as a disclosed gold-price proxy), opt_02/opt_03 (extra calculate calls producing a real, correctly-computed PCR).
- One genuine failure: fund_05.

Real findings

Quota/key management gaps (not bugs in scoring logic, but real operational discoveries):

- `_key_index` always resets to 0 on every fresh script execution no rotation state persists across runs, so a fresh run after hitting quota retries the SAME exhausted key first unless `.env` is manually reordered. `DailyQuotaExhausted` never triggers reactive rotation to the next configured key — only the proactive `QUESTIONS_PER_KEY` counter rotates, and that counter also resets to 0 per run.
- Adversarial questions cost far more than `QUESTIONS_PER_KEY=8` assumes, since it counts questions, not LLM calls, and those aren't 1:1. `fund_05` alone consumed roughly the budget of 3-4 clean questions.

Bug found — cap-recovery returned literal blank answers

`fund_05` (ambiguous Tata Motors ticker) hit `MAX_ITERATIONS` after failed attempts on `TATAMOTORS.NS` and guessing unrelated `TATASTEEL.NS`, instead of flagging ambiguity. The recovery call in `ask()` returned no content blocks; `extract_text()` fell through to `str([])`, displaying as `[]`. Additionally, recovery text was discarded instead of returning or persisting the recovered result.

Scorer limitation: `fund_05` scored `recall=1.0` because calling `get_company_overview` on the wrong company satisfied the name-only check, showing high recall doesn't ensure accuracy.

Patch applied — Round 1 (cap-recovery empty-answer fix)

In `ask()`'s cap-recovery block: detect when the recovery call's answer text is empty or literally `[]`, and fall back to a deterministic, honest message built from `TOOL_CALL_LOG` (listing which tools were tried) instead of showing nothing. Also fixed the return-value bug so a real recovered answer is persisted to checkpointed state and returned to the caller.

Segment 4 — Remaining Batches (filing, regulation, multi_hop)

Goal

Finish the eval set, watching specifically for repeats of Round 1's failure mode, and testing whether the patch holds up under real conditions.

filing (6/6) and regulation (6/6) complete in two runs.

multi_hop (6/6) complete across five separate runs.

Output

- Recall 1.000 in every category.
- Regulation finished with zero real failures — its worst cases (reg_05, reg_06) were both good-faith over-searching with correct final answers, consistent with Part 2's RAGAS finding that regulation retrieval was already meaningfully stronger than filing retrieval. filing and multi_hop carried the real problems.

Real findings, in the order they were found

- **Round 1's bug reproduced** — filing_04, worse than fund_05. Same cap-hit pattern, but this time the correct answer (TCS's workforce = 593,798) had already been retrieved TWICE (iterations 3 and 5) before the cap forced a discard. This confirmed the bug wasn't a one-off and that its root cause, re-querying past a sufficient answer — was actively destroying correct, already-verified data, not just wasting quota. Round 1's fix verified live, but its limits exposed — filing_05. The single most important honesty check in the eval set (Q3-vs-Q1-FY27 period mismatch) hit the cap again, and this time produced the graceful fallback message instead of "[]" — the patch working exactly as designed. But the fallback message itself never delivered the actual disclosure the question needed; it discarded the correct 533M-subscriber figure that was sitting in the very first search result and just apologized instead. The patch prevents silence; it does not, on its own, produce a good answer.
- **Round 2's mechanism confirmed working** — multi_02. Hit the cap, first synthesis came back empty again, and the round-2 digest retry (added after filing_05) produced real text instead of falling to the apology — first live proof the rescue mechanism functions at all. But it surfaced a new, more specific bug: the rescued text FALSELY claimed no SEBI regulation information was found, despite the correct contract-size regulation sitting in TOOL_CALL_LOG. Root cause: round-2's blind 400-character truncation of raw JSON strings cut off the actual answer-bearing sentence before reaching it. A less certain variant of the same failure — multi_03 (pre-fix). Rescue claimed "no information" despite the digest containing PE, RoA, NIM, price, book value, and a computed P/B ratio — genuinely enough to answer well, and small enough that truncation alone doesn't explain it. Suggested the rescue call could misjudge a digest full of real numbers as empty, not only lose data to truncation.

- A distinct, partial-success shape — multi_04. Rescue produced factually CORRECT content (no false claims at all — contract-size, premium-collection, and calendar-spread dates were all right) but never answered the actual interpretive question asked ("would this affect what I'm trading") — it listed measures instead of reasoning about the user's specific situation. Different enough from both "real failure" and "rigor" to need its own category (partial) in the aggregate report.
- A structurally different bug — multi_05 (pre-fix). A single search_filings call returning 5 individually-scored sub-results (relevance 5.251 down to 1.075) filled almost the entire round-3 digest budget on its own, crowding out the get_option_chain and calculating results that were also needed — even though those were small and directly relevant. The rescue correctly reported ICICI's NIM but falsely claimed no NIFTY option/OI data existed, despite it being fully present in TOOL_CALL_LOG.

Patch applied

- Round 2 (digest-based rescue attempt): Before falling back to the apology message, try one more short, directive synthesis call using a compact digest of TOOL_CALL_LOG's raw results (truncated to ~400 characters per call, ~3000 total) — on the theory that a shorter, more targeted prompt is more likely to succeed than repeating the same full context that just failed.
- Round 3 (JSON-aware, relevance-sorted digest): Replaced blind raw-JSON truncation with: parse each tool result, extract actual "text" fields from search_filings/search_regulations results (falling back to raw truncation for compact tool types like price/overview/calculate), and sort all candidate snippets by relevance_score so the limited digest budget goes to the most likely- relevant content first. Verified live on multi_03: re-run after the patch went from recall=1.0, precision=0.333 (false "no information" claim) to a clean 1.0/1.0 — the cap wasn't even hit on re-run, and the answer included real historical context, a comparison table, and a defensible conclusion.
- Round 4 (guaranteed slot per tool call): multi_05 showed a different problem: one tool call's multiple sub-results could still crowd out every OTHER tool call's data by sheer count, even at equal or higher real relevance. Fixed by guaranteeing each distinct tool call contributes at least one snippet (its own highest-relevance sub-result, or its whole compact output for non-search tools) before any remaining digest budget is filled by the best leftovers globally. Verified live on multi_05: re-run after the patch correctly stated BOTH ICICI's NIM (4.36%) and real NIFTY OI figures (716,411 calls / 615,044 puts) — no false claims — with an honest, non-fabricated note that the full comparison itself wasn't finished (classified partial, not a failure).

Segment 5 — Aggregate Report

Goal

One script that separates genuine agent failures from misleadingly-scored "extra rigor," since raw recall/precision numbers alone conflate the two — and, as it turned out, conflate a third shape (partial) as well.

Concepts covered

- **Hand-annotated qualitative notes, not just recomputed numbers:** The scorer can only compare tool names, it cannot tell "unnecessary call" apart from "call that made the answer better" (opt_02), or "matched tool name" apart from "matched tool name, wrong argument" (fund_05's TATASTEEL.NS). Every question scoring below 1.0 on either metric was read manually and tagged before being included in any summary.
- **Three categories, not two:** rigor (good behavior, misleading score) and real_failure (genuine problem) weren't enough — multi_04 and multi_05 needed a third, partial: facts correctly rescued, but the actual question not fully answered.
- **Deduplication by question id, keeping the latest occurrence:** The results file is append-only; re-running a question with --force (as happened for multi_03 and multi_05) adds a new line rather than replacing the old one. Without deduping at load time, a re-scored question would be counted twice, once with stale score, once with its current one, corrupting that category's average. Fixed by keeping only the last-seen entry per id.

Files

aggregate_report.py → Reads tool_eval_results.jsonl (deduped), produces tool_eval_report.md

Command:

```
python aggregate_report.py
```

Output

Category	Number of Questions	Avg Recall	Avg Precision
filing	6	1.000	0.525
fundamentals	5	1.000	0.840
multi_hop	6	1.000	0.567
option_chain	5	1.000	0.767
price	5	1.000	0.800
regulation	6	1.000	0.756

Overall: recall 1.000, precision 0.701

15 of 33 questions scored below 1.0 on at least one metric. Broken down by what actually happened, not just the raw number:

- **4 real failures:** fund_05, filing_04, filing_05, multi_02 — all four traceable to the same underlying cause (over-searching past a sufficient answer → iteration cap → answer lost or degraded during recovery), at different stages of the patch history.
- **2 partial:** multi_04, multi_05, facts correctly recovered, actual question not fully answered.
- **9 rigor cases:** price_05, opt_02, opt_03, filing_02, filing_03, reg_05, reg_06, multi_01, multi_06 — extra tool calls that produced better, not worse, answers.
- **1 resolved-by-later-patch:** multi_03's original score reflected a pre-Round-3 failure; its re-run after the fix scored a clean 1.0/1.0 and no longer appears in the imperfect-scores table at all.

filing remains the weakest category by precision (0.525), consistent with Part 2's RAGAS finding that filings retrieval scored meaningfully lower than regulations (0.375/0.250 vs. 0.769/0.800 precision/recall) — a pattern that held up independently across two entirely different scoring methods.

A complete, deduped, re-runnable aggregate report — the actual deliverable Part 3's original "aggregate full report" step called for — that correctly separates real agent failures (4), incomplete-but-honest rescues (2), and scorer artifacts (9) across all 33 questions.

Output

A complete, live-verified tool-call correctness evaluation across all 33 questions and 6 categories, with recall at a clean 1.000 throughout and precision losses fully explained rather than left as raw numbers — 4 real failures, 2 honest partial answers, 9 cases of good-faith extra rigor, and 1 confirmed fix. Along the way, found and live-verified fixes for 4 distinct, increasingly specific bugs in agent.py's cap-recovery path, each one only discoverable by actually running real questions against real data — none of the four would have been caught by unit-testing the scorer alone, and each fix's necessity was demonstrated by a real live failure before being written, not guessed at in advance.

Bugs found & fixed in agent.py — cross-cutting summary

#	Bug	Root cause	Fix	Live verification
1	Cap-recovery could return the literal string "[]" as the final answer, and even a real recovered answer was discarded	extract_text() falls through to str([]) when the wrap-up LLM call returns zero content blocks; the recovered answer, when non-empty, was also never persisted or returned	Detect empty/"[]" text, fall back to a deterministic honest message listing tools tried; fixed the return-value bug so real recovered answers are persisted and returned	filing_05 — graceful message shown instead of blank
2	Round-1's fallback discarded already-correct gathered data instead of trying to use it	The apology message replaced the answer entirely rather than reading TOOL_CALL_LOG's contents	Before falling back, attempt one more short synthesis call using a compact digest of raw tool results	multi_02 — mechanism confirmed producing real text, not a blank/apology
3	Round-2's digest lost real answers to blind truncation, or was misjudged as empty despite being populated	400-raw-character-per-call truncation frequently cut off answer-bearing sentences buried inside JSON	Parse each result, extract actual text fields, sort by relevance_score, prioritize highest-relevance content within the digest budget	multi_03 — re-run went from a false "no information" claim (precision 0.333) to a clean, correct answer (1.0/1.0)
4	Round-3's relevance sort let one tool call's multiple sub-results crowd out every other tool call's data entirely	A single search_filings call with 5 scored sub-results could fill nearly the whole digest budget, even against smaller, equally-relevant results from other tools	Guarantee every distinct tool call at least one digest slot (its own best sub-result) before filling remaining budget by global relevance	multi_05 — re-run correctly included both the filing fact (NIM) and the option-chain fact (OI), previously only one or neither

Key Rotation: zero-cost test (no API calls needed)

All 4 isolated tests passed: persistence across simulated runs, stale-index discarding, reactive rotation (caught DailyQuotaExhausted, rotated to GEMINI_API_KEY2, retried successfully), and correct re-raise when no keys remain. Confirmed working as designed.

Stop-searching prompt: requires a live test

Re-ran filing_02 and filing_04 with --force after applying the prompt patch.

Question	Before patch	After patch
filing_02	4 calls, precision 0.25	1 call, precision 1.0
filing_04	5 calls, hit the cap	2 calls, no cap hit

filing_04 in particular — the question responsible for two separate bugs found earlier in Part 3 — now resolves cleanly without ever reaching the iteration cap. Confirms the prompt tightening addresses the actual root cause (over-searching), not just its symptoms.

Tradeoff observed: filing_02's post-patch answer is correct but thinner than its pre-patch version (current-quarter NIM only, vs. a full 6-quarter trend table before). Stopping after the first sufficient result trades some depth for reliability — still fully correct for what was asked, but worth watching if similar depth loss shows up on other questions.

Week 6 Notes

Week 6, Part 1 — Citation Enforcement + Graceful Refusal

Goal

Make "no claim without a source" and "say so instead of guessing" enforceable. The task is to stop trusting the system prompt alone to make the model behave, and add a verification layer that checks what actually happened against TOOL_CALL_LOG before an answer ships.

Concepts covered

- Prompt instruction vs. verification layer — a system prompt asking for citations is necessary but not sufficient; models can comply, partially comply, or ignore it. The real guarantee has to come from code that inspects the tool log after the fact, not from asking nicely.
- Ground truth from TOOL_CALL_LOG, not from the model's own claims — whether a source was actually retrieved, and what its real filename/company is read directly from the logged tool results, the same log already used.
- Two independent failure modes — (a) a claim made without naming its source, and (b) a claim made when the underlying search actually returned nothing. These require different fixes and are checked separately.
- Auditable-but-loose text matching — the citation check doesn't require an exact "(Source: ...)" string; it checks whether the real source filename (or a long, unambiguous chunk of it) appears anywhere in the answer. Strict on substance (a real, traceable document must be named), loose on formatting (the model's exact phrasing can vary).

Files

agent_loop/agent.py →

Added RAG_TOOL_NAMES, NO_DATA_PHRASES, _rag_calls_this_turn(), _answer_cites_sources(), _answer_acknowledges_no_data(), and enforce_citations_and_refusals(). Extended SYSTEM_PROMPT with the required citation format and the no-guessing instruction. Wired into both exit paths of ask() — the normal completion and the cap-recovery fallback.

Commands

python agent.py

Output

(live, real data)

1. Question: "What did Reliance's latest filing say about Jio's subscriber growth?"
 - search_filings returned 5 real chunks from Media_Release_RIL_Q1_FY2026-27_Financial_and_Operational_Performance.pdf.
 - The first draft already cited every bullet correctly — enforcement checked, found nothing wrong, made no extra call. Confirms the check is a true no-op when the model already complies, not a blanket rewrite.
2. Question: "What did TCS's filing say about their crypto mining strategy?"
 - search_filings returned two consecutive {"error": "No relevant filings found..."} results (the model retried once with reworded args before giving up).
 - The draft answer already said "No matching filing was found regarding TCS's crypto mining strategy" — enforcement against a no-op, correctly recognizing an honest refusal needs no repair.
3. Question: "What is TCS's current stock price?"
 - No RAG tool called at all — confirmed zero [citation enforcement] activity and zero extra API cost on non-RAG turns.

Output

Verification layer that inspects TOOL_CALL_LOG rather than trusting the model's compliance, confirmed on real data in three states: silently correct when the draft already cites properly, silently correct when the draft already refuses honestly, and — after finding and fixing a real gap where a bare company-name mention was wrongly accepted as a citation — actively firing and repairing when the draft genuinely lacks a traceable source. Zero added cost on non-RAG turns.

Week 6, Part 2 — F&O Disclaimer + Hard Tool-Call Cap

Goal

Two independent requirements:

1. every answer surfacing real option-chain/OI data must carry a clear "research, not a trading recommendation" disclaimer
2. A hard ceiling on total tool executions per question, distinct from the existing MAX_ITERATIONS reasoning-step cap

Concepts covered

- Deterministic guardrail vs. LLM-repaired guardrail — Part 1's citation fix needed an LLM call because the specific wording (which source, which claim) genuinely varies per answer. The F&O disclaimer is fixed boilerplate — no rephrasing needed — so it's applied as a plain string append, not another model call. Cheaper and can't fail/hallucinate the way a repair call theoretically could.
- MAX_ITERATIONS vs. MAX_TOOL_CALLS_PER_QUESTION are answering different questions — the former caps how many times the model gets to reason (agent-node turns); the latter caps how many real tool executions happen in total, since a single reasoning turn can request several tool calls at once. A question could stay well under the iteration cap while still running an unbounded number of actual tool calls without this second cap.
- Guardrails composed in one entry point - apply_guardrails() now runs citation/refusal enforcement first, then the disclaimer check, against whatever text results — so the disclaimer is guaranteed to land on the final shipped text even if Part 1 rewrote it.

Files

agent_loop/agent.py →

Added MAX_TOOL_CALLS_PER_QUESTION = 15 (separate from MAX_ITERATIONS = 6).

Added OPTION_TOOL_NAMES, DISCLAIMER_TEXT, DISCLAIMER_PHRASES, _option_data_used_this_turn(), and ensure_fo_disclaimer(). Added apply_guardrails() as the single entry point combining Part 1 + Part 2 checks, replacing direct calls to enforce_citations_and_refusals() at both exit paths. Added the tool-call budget check inside call_tools(), refusing (not silently dropping) calls once the budget is reached

Commands

python agent.py

python -c "import agent; agent.MAX_TOOL_CALLS_PER_QUESTION = 1; agent.ask('Compare TCS and Infosys P/E, and check NIFTY option activity', thread_id='cap-test')"

Output

(live, real data)

1. Question: "Is there heavy put writing near the current NIFTY strike?"
 - `get_option_chain` returned real near-ATM strike data.
 - Final answer included a full OI breakdown table and correctly identified heavy put OI at 24,200 (258,464) and 24,000 (263,214) as support, call OI at 24,300 (285,330) as resistance - and ended with the disclaimer: "This information is for research purposes only and is not a trading recommendation. Please do your own due diligence or consult a financial advisor before making any trading decisions."
2. Question: "TCS Price"
 - `get_stock_price` only — no option data, no disclaimer appended. Confirms `ensure_fo_disclaimer()` is a true no-op outside F&O answers.

Forced cap test — `MAX_TOOL_CALLS_PER_QUESTION` set to 1, question needing 3 tool calls ("Compare TCS and Infosys P/E, and check NIFTY option activity"):

- `get_company_overview(TCS.NS)` executed (1st call, budget reached).
- `get_company_overview(INFY.NS)` refused: [tool-call cap] Refusing `get_company_overview({'ticker': 'INFY.NS'})` — budget of 1 tool call reached this question.
- `get_option_chain(NIFTY)` refused the same way.
- Final answer reported TCS's real P/E (16.42) and market cap (₹8,20,256.65 Cr), then explicitly stated Infosys's data and NIFTY option data could not be retrieved due to the budget limit — no hallucinated numbers for either. No disclaimer appended, correctly, since the refused option-chain call never returned real data.

Output

A deterministic, zero-extra-cost F&O disclaimer confirmed live on real option-chain data (and confirmed absent on non-F&O answers), plus a genuinely new hard cap on total tool executions per question — distinct from the existing iteration cap — verified both offline (boundary case) and live (forced cap=1 test), with the model degrading gracefully and honestly rather than hallucinating when the budget cut off two of three needed tool calls.

Week 6, Part 3: Streamlit UI + Pre-Deployment Testing

Goal

The terminal-only agent into something reachable by anyone with a browser link — a two-view Streamlit app. Split into two segments: building the UI itself, and a pre-deployment check that surfaced and fixes real problems in the UI-building phase.

Segment A — Streamlit UI (deploy/app.py)

Concepts

- Thin UI, one agent implementation — app.py only calls agent.ask(), so all fixes and logic are shared automatically.
- Session-based thread_id — one shared agent, but each user gets a unique UUID to keep chats separate.
- Tested with Streamlit AppTest — mocked agent + automated UI/session-state tests before using real quota.
- Fixed chat layout — used a fixed-height container to prevent Streamlit reruns from resetting the scroll.
- Stable CSS hook — targeted st.container(key=...) with CSS for precise styling.
- Header overlap handled — balanced top padding so content stays clear of Streamlit's fixed header.

Files

deploy/app.py →

The entire UI. Two tabs (Cash Equity Research, F&O/OI View), sidebar watchlist (clickable), session-scoped thread_id, injected CSS for layout/scroll fixes.

Commands

```
cd deploy
```

```
streamlit run deploy/app.py
```

Output of Segment A

A working two-tab Streamlit app, verified structurally via 13 automated AppTest checks (initial load, both tabs, sidebar, example prompts, both failure-handling paths, session isolation, watchlist-click routing, multi-turn container stability) and confirmed live against real Gemini/RAG data.

Segment B — Pre-Deployment Testing Checklist

Goal

The deployed environment will actually have everything the local one has, before finding out the hard way on a live Streamlit Cloud deploy. Local testing alone only proves the code is correct — it doesn't prove requirements.txt is complete, that secrets will load the way Streamlit Cloud provides them, or that the vector store will even be present once GitHub is the only source of truth.

Concepts covered

- Clean checkout testing - tested in a fresh venv setup with existing installs.
- Secrets parity - confirmed Streamlit Cloud secrets work with the existing load_dotenv() + os.environ.get() setup.
- Data dependency check - made sure rag/chroma_db is tracked and available after deployment.
- Session isolation - verified that separate browser sessions keep conversations independent, even with a shared LangGraph app and checkpoint.

Command

- .gitignore / ChromaDB check
Ran git check-ignore -v and checked the ChromaDB directory size.
- Fresh environment install test
Created a fresh virtual environment and ran pip install -r requirements.txt, followed by python -c "import agent".
- Secrets loading test
Checked st.secrets.toml for all required GEMINI_API_KEY entries, then temporarily renamed the .env file to confirm that secrets were loading correctly.
- Terminal regression test
Ran four test questions covering price, filing, regulation, and option-chain queries through python agent.py.
- Full Streamlit functional test
Tested both tabs, the watchlist, multi-turn memory and scrolling, the F&O disclaimer, and starting a new conversation.
- Two-session concurrency test
Opened two incognito browser windows and confirmed that each session maintained an independent thread_id and conversation.
- Quota budget check
Reviewed TOOL_CALL_LOG using Group-Object key_alias and confirmed that usage stayed below the 20/day ceiling across all six price calls.